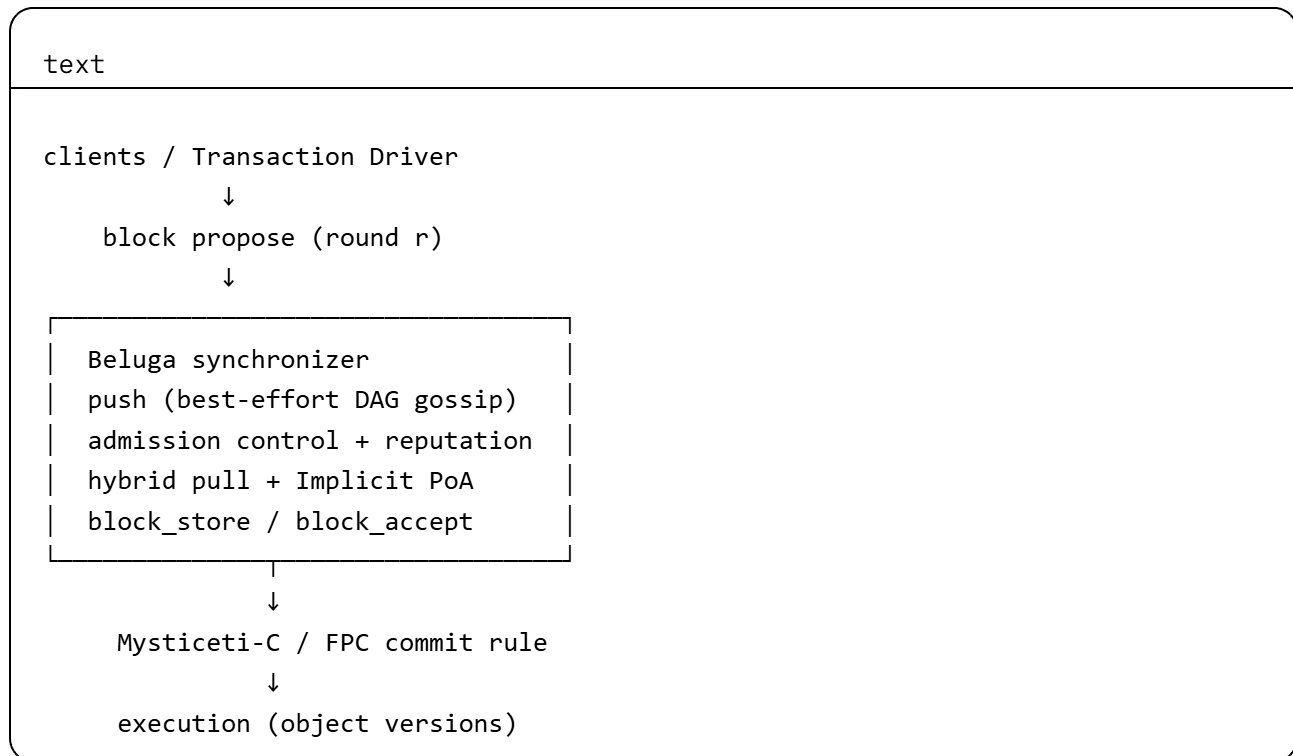


Beluga is not another consensus protocol. It is the **block dissemination layer under Mysticeti**: how validators get the bytes of DAG vertices when the happy-path gossip fails. Mysticeti's commit rule is cheap *because* blocks are uncertified. That makes missing parents lethal — and it makes naive “just pull the hash” a denial-of-service primitive. Beluga is the scarcity-aware fix. It has been on Sui mainnet since `v1.42.0` (January 2025).

Where it sits



Consensus **orders** accepted blocks. Execution **runs** stored blocks. Beluga's job is: every honest node that needs block B eventually has B , without all honest nodes fetching B from everyone at once.

Mysticeti-Beluga does **not** change the 3-round commit rule. It changes **which parents you are allowed to cite** and **how you fetch holes**.

Why a dedicated synchronizer

Happy path in an uncertified DAG:

1. Author signs one block, best-effort broadcasts it ($\approx \delta$ latency).
2. Peers accept it and cite it next round.
3. Stop retransmitting once you move to the next round.

That is optimal when the network is fine. It is *not* reliable broadcast. Under delay or a Byzantine author you get **holes**: you see a block that lists parent hashes you do not have. Classic reaction: pull those hashes from whoever mentioned them.

The paper's attack is **pull induction**:

- A faulty validator (or a coalition) publishes blocks whose ancestor lists point at **rare or fake-looking** refs.
- Honest nodes independently pull the same missing set from many peers.
- Bandwidth collapses; rounds stall; tail latency explodes.

Uncertified DAGs are especially exposed because there is no QC that already proves “ $2f + 1$ nodes stored this.” Sui's pre-Beluga push-pull hit this under network-degradation attacks. After Beluga, production tail latency under those attacks dropped about **5×**; lab attack scenarios report up to **3× throughput / 25× latency** vs the old synchronizer.

Abstraction

A **block synchronizer** is a module with roughly:

Call	Meaning
<code>block_propose(B, r)</code>	I authored this round- r block; push it
<code>block_store(B)</code>	Bytes are local; execution may use B once ordered
<code>block_accept(B)</code>	B is eligible as a DAG parent / vote
pull / header exchange	Incremental retrieval of missing refs

Beluga implements that with two halves: **admission-controlled push** and **hybrid pull**.

1. Optimistic push + admission control

Push stays best-effort DAG gossip (one signature, δ -class latency). Retransmit budget is bounded: once the author advances rounds, they stop flooding that block.

What changes is **which blocks you allow into the next proposal**.

Reputation. Each validator scores others from observed behavior (invalid blocks, unanswered pulls, induced missing parents, timeouts). Scores persist across rounds.

Admission control (AC) when you pick ancestors for round r :

- Prefer **leader / anchor** blocks and the $2f + 1$ supporters needed for the commit rule.
- Prefer authors above a reputation threshold R_t .
- Deprioritize or exclude low-reputation authors from parent sets so their junk does not become a pull magnet.

Round advancement in Mysticeti-Beluga **intersects** Mysticeti's quorum-clock with AC: you move to r when you have $2f + 1$ accepted round- $r - 1$ blocks **and** (in the robust variant) those include a high-reputation leader and enough high-reputation supporters. That keeps liveness of the commit rule while starving pull-induction parents.

Happy-path push latency can sit arbitrarily close to 2Δ if the reputation window R_L is large enough — i.e. you do not pay extra rounds when everyone is honest.

2. Hybrid pull and Implicit Proof-of-Availability

When a hole remains, Beluga does **not** “ask every peer for every hash.”

Implicit Proof-of-Availability (ImPoA). If $2f + 1$ later blocks (from well-reputed authors) already **cite** B , honest nodes treat that pattern as evidence that B is available *somewhere in the honest set* — the same structural idea as Mysticeti's implicit certificates, applied to storage. You then pull from a **small, scored set of holders**, not from the whole committee.

Hybrid pull:

1. Try a short, parallel pull from the **citers** with best reputation / RTT.
2. If that fails, widen the set, but **rate-limit** outstanding pulls per missing block and per peer.
3. Deduplicate: one in-flight fetch per hash cluster-wide from this node's point of view.
4. Bad responders lose reputation → they are asked less and cited less, which bounds amplification.

So recovery cost is **O(missing blocks × small fanout)**, not $O(\text{missing} \times n^2)$ induced pulls. That is the “scarcity-aware” part: bandwidth is the scarce resource the adversary was burning.

Interface back into Mysticeti

text

Beluga accepts block → consensus may use it as parent / vote
Beluga stores block → after C/FPC commit, execution can run txs

Ordering logic stays Mysticeti (anchors, skip vs certificate patterns, FPC votes in the DAG). Security proofs for Mysticeti-Beluga show prefix-consistent honest outputs — Beluga does not invent a new commit rule; it supplies the availability Mysticeti assumed. Lean formalization covers the non-probabilistic pieces.

Implementation: Rust inside the Mysticeti stack, Tokio, raw TCP reliable point-to-point channels (no RPC framework in the paper’s prototype).

What it does *not* do

- It does not certify every block like Narwhal (that would bring back CPU and rounds).
- It does not order transactions (no fee auction, no STM).
- It does not remove MEV from DAG proposal order.
- It cannot make a missing **Byzantine author’s** unique payload appear if no honest node ever received it — it only guarantees availability of blocks that honest commit rules will actually depend on.

Why it belongs next to Mysticeti in a game chain

Uncertified DAG + object fast path is how you get sub-second owned transfers. That design **moves availability out of the certificate and into the synchronizer**. Without something like Beluga, an attacker targeting holes in the DAG turns your low-latency consensus into a pull storm — exactly when a metaverse economy is busiest.

Stack to remember:

Layer	Job
Transaction Driver	get a tx into one validator
Beluga	make DAG blocks available without pull amplification
Mysticeti-C / FPC	decide commit / skip / fast cert
Object executor	run Move on owned vs shared versions

Beluga is the piece that makes the uncertified DAG **operable under attack**, not just fast in

a datacenter